бюджетное профессиональное образовательное учреждение
Вологодской области
«Череповецкий лесомеханический техникум им. В.П. Чкалова»

Специальность 09.02.07 Информационные системы и программирование

ДИПЛОМНЫЙ ПРОЕКТ

РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА АНАЛИЗА И ГЕНЕРАЦИИ СЕТЕВОГО ТРАФИКА

СТУДЕНТ: 4 курса группы ИС-42
Неклюдов Никита Сергеевич

НАУЧНЫЙ РУКОВОДИТЕЛЬ: _____
преподаватель
Табунов П.А.

РЕЦЕНЗЕНТ: _____

ДОПУЩЕНО К ЗАЩИТЕ: «___» _____ 2026г.

ДАТА ЗАЩИТЫ: «___» _____ 2026г.

ЗАВ. ОТДЕЛЕНИЕМ:

ОЦЕНКА ЗАЩИТЫ: _____

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТРЕБОВАНИЙ 7
1.1 Характеристика предметной области: сетевой трафик и протоколы 7
1.2 Обзор существующих средств анализа и генерации 9
1.3 Постановка задачи и требования к функционалу 11
1.3.1 Требования к анализу и мониторингу трафика 12
1.3.2 Требования к генерации сетевых пакетов 12
1.3.3 Требования к фильтрации и хранению данных 12
1.4 Требования к безопасности и этике использования 13
1.4.1 Предотвращение несанкционированного использования (Misuse) 13
1.4.2 Этические аспекты и предотвращение сетевых атак 14
1.5 Выбор средств разработки 14
2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА 17
2.1 Архитектура приложения и модульная структура 17
2.2 Моделирование вариантов использования системы 18
2.3 Проектирование классовой модели 20
2.4 Моделирование процессов обработки пакетов 23
2.5 Алгоритмы генерации и анализа трафика 24
2.6 Структура данных и хранение результатов 26
2.7 Проектирование интерфейса взаимодействия 27
3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ 30
3.1 Программная реализация классов и методов 30
3.2 Работа с сетевыми сокетами и сырыми пакетами 32
3.3 Реализация механизмов фильтрации и статистики 34
3.4 Тестирование функциональности 37
3.5 Оценка производительности и нагрузки на систему 40
4. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ 42
4.1 Расчет трудозатрат на разработку и отладку программы 42
4.2 Расчет себестоимости программного продукта 43
4.3 Расчет цены программного продукта 44

4.4 Расчет экономической эффективности 45
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ 47
ПРИЛОЖЕНИЕ А 49
ПРИЛОЖЕНИЕ Б 53
ПРИЛОЖЕНИЕ В 59

ВВЕДЕНИЕ

На сегодняшний день функционирование любой современной организации невозможно без использования локальных и глобальных вычислительных сетей. С развитием технологий «интернета вещей» (IoT), облачных вычислений и распределенных систем, объемы передаваемого трафика растут в геометрической прогрессии. В связи с этим, вопросы обеспечения стабильности, производительности и безопасности сетевой инфраструктуры выходят на первый план.

Мониторинг и глубокий анализ сетевого трафика являются базовыми процессами в работе системного администратора и специалиста по информационной безопасности. Инструментарий для выполнения этих задач должен быть не только мощным, но и гибким. Существующие на рынке программные решения часто обладают рядом недостатков для специфических задач. Например, профессиональные анализаторы уровня Wireshark обладают избыточным функционалом, который требует значительных системных ресурсов и длительного освоения, а консольные утилиты вроде tcpdump не всегда обеспечивают необходимую наглядность и интерактивность при генерации специфических сетевых пакетов.

Особое значение приобретает возможность создания собственных инструментов на базе высокоуровневых языков программирования. Выбор языка Python и библиотеки Scapy для реализации дипломного проекта обусловлен их способностью работать с «сырыми» сокетами (Raw Sockets), что позволяет формировать пакеты произвольной структуры. Это критически важно не только для пассивного анализа, но и для активного тестирования сетевого оборудования на устойчивость к различным типам нагрузок. Таким образом, разработка программного комплекса, совмещающего в себе функции мониторинга и гибкой генерации трафика, является актуальной задачей, отвечающей современным требованиям к инструментам сетевого администрирования.

Объект исследования: процессы передачи и обработки пакетов данных в вычислительных сетях на базе стека протоколов TCP/IP.

Предмет исследования: программные алгоритмы и методы перехвата, декодирования и формирования пакетов данных с использованием специализированных библиотек анализа трафика.

Цель работы: проектирование и программная реализация многофункционального комплекса для мониторинга и формирования сетевого трафика, предоставляющего пользователю инструменты для детального разбора пакетов и тестирования сетевых соединений.

Для достижения поставленной цели в рамках дипломного проекта необходимо решить следующие задачи:

1. Выполнить подробный анализ предметной области, изучив теоретические аспекты функционирования модели OSI и стека TCP/IP.
2. Провести сравнительный обзор существующих программных средств анализа трафика, выделив их сильные и слабые стороны для обоснования необходимости разработки собственного решения.
3. Обосновать выбор технологического стека (Python, Scapy, Npcap) и среды разработки PyCharm как наиболее эффективных инструментов для решения поставленных задач.
4. Разработать детальное техническое задание на создание программного продукта в соответствии с требованиями ГОСТ 19.201-78.
5. Спроектировать архитектуру программного комплекса, подготовить полный пакет UML-диаграмм (классов, последовательности, деятельности), описывающих внутреннюю логику работы приложения.
6. Реализовать программный код модулей захвата, анализа и генерации сетевых пакетов.
7. Провести серию испытаний разработанного средства в контролируемой сетевой среде, выполнить отладку и оценку соответствия функционала заявленным требованиям.

8. Оформить пояснительную записку и подготовить демонстрационные материалы для защиты проекта.

Научная новизна работы заключается в формировании концепции программного средства, сочетающего возможности анализа и генерации сетевого трафика в рамках единого интерфейса с акцентом на удобство использования и наглядность представления данных.

1. В работе применяются следующие методы:
2. теоретический анализ научной литературы и документации
3. сравнительный анализ существующих решений
4. моделирование и проектирование системы с использованием UML

Теоретическая значимость работы заключается в систематизации знаний о методах анализа и генерации сетевого трафика. Практическая значимость состоит в возможности использования результатов проектирования в учебных целях и при изучении сетевых технологий.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТРЕБОВАНИЙ

1.1 Характеристика предметной области: сетевой трафик и протоколы

Функционирование современных информационных систем неразрывно связано с обменом данными через компьютерные сети. Под сетевым трафиком понимается совокупность данных, передаваемых по каналам связи в определенный промежуток времени. Понимание природы этого трафика, его структуры и механизмов передачи является фундаментом для разработки средств мониторинга и генерации пакетов.

В основе сетевого взаимодействия лежит эталонная модель взаимодействия открытых систем (OSI), которая разделяет процесс передачи данных на семь логических уровней. Для разработки программного комплекса мониторинга наиболее критичными являются первые четыре уровня, так как именно на них происходит формирование структуры кадров и пакетов, с которыми работает библиотека Scapy:

1. Физический уровень (Physical): определяет физические характеристики каналов связи. На этом уровне наш программный комплекс взаимодействует с сетевым адаптером, переведенным в «смешанный режим» (promiscuous mode), что позволяет захватывать все пакеты, проходящие через сегмент сети, а не только адресованные конкретному узлу.
2. Канальный уровень (Data Link): здесь данные оформляются в кадры (frames). Основным протоколом является Ethernet. В рамках дипломного проектирования важно учитывать структуру Ethernet-кадра, включающую MAC-адреса отправителя и получателя, а также поле типа протокола верхнего уровня.
3. Сетевой уровень (Network): отвечает за маршрутизацию данных. Ключевым протоколом здесь выступает IP (Internet Protocol). Для анализатора трафика принципиально важно уметь разбирать IP-

заголовки, извлекая информацию об адресации, времени жизни пакета (TTL) и контрольных суммах.

4. Транспортный уровень (Transport): обеспечивает доставку данных между процессами. Основные протоколы − TCP (с установлением соединения) и UDP (без установления соединения). Разрабатываемое программное средство должно корректно идентифицировать порты приложений и анализировать флаги TCP-пакетов, что необходимо для диагностики состояния сетевых служб.

Сравнительные характеристики основных транспортных протоколов представлены в таблице 1

Таблица 1 – Сравнение протоколов TCP и UDP

Параметр	TCP	UDP
Тип соединения	С установлением соединения	Без установления соединения
Надёжность	Высокая	Низкая
Контроль доставки	Есть	Отсутствует
Порядок пакетов	Гарантируется	Не гарантируется
Скорость передачи	Ниже	Выше
Область применения	Веб, почта, файлы	Стриминг, игры, VoIP

Стек протоколов TCP/IP, являющийся практической реализацией модели OSI, накладывает свои особенности на структуру трафика. Каждый пакет представляет собой многослойную структуру, где данные одного уровня инкапсулируются в заголовки другого. Процесс анализа трафика (сниффинг) по сути является процессом деинкапсуляции − последовательного «снятия» заголовков для изучения содержимого на каждом этапе.

Генерация трафика, в свою очередь, представляет собой обратный процесс. В дипломном проекте упор делается на создание инструмента, который позволит программно формировать пакеты, начиная с нижних уровней. Это дает возможность симулировать различные сетевые ситуации: от

стандартных запросов до специфических нагрузочных тестов. Использование языка Python позволяет автоматизировать этот процесс, делая его более гибким по сравнению с жестко заданными шаблонами в готовых утилитах.

Таким образом, предметная область охватывает широкий спектр технологий передачи данных. Глубокое понимание структуры заголовков и логики инкапсуляции протоколов TCP/IP является необходимым условием для реализации алгоритмов захвата и формирования трафика, которые будут подробно рассмотрены в последующих разделах работы.

1.2 Обзор существующих средств анализа и генерации

Для обоснования необходимости разработки собственного программного комплекса необходимо провести критический анализ наиболее востребованных инструментов, представленных на рынке программного обеспечения для сетевого администрирования. Основными критериями оценки выступают: функциональность, доступность, удобство интерфейса и возможность кастомной генерации пакетов.

В качестве объектов сравнения выбраны три наиболее репрезентативных средства: Wireshark, tcpdump и библиотека Scapy.

1. Wireshark. Является стандартом в области графического анализа трафика. Программа обладает мощнейшим функционалом по декодированию сотен протоколов и предоставляет удобные средства фильтрации (Display Filters). Однако высокий порог вхождения и значительное потребление ресурсов памяти делают его использование не всегда оправданным для локальных задач или автоматизированных тестов. Основной минус для нашей темы − отсутствие встроенного удобного конструктора для массовой генерации специфических пакетов «из коробки».
2. Tcpdump. Легковесная консольная утилита, работающая практически во всех Unix-подобных системах. Она незаменима для быстрой

диагностики на серверах без графической оболочки. Тем не менее, отсутствие визуализации и сложность анализа больших потоков данных в текстовом виде ограничивают её применение в образовательных целях и при проектировании сложных сетевых сценариев.

3. Scapy. Это мощная интерактивная библиотека на языке Python. В отличие от предыдущих инструментов, Scapy позволяет не только захватывать пакеты, но и «собирать» их вручную слой за слоем. Это дает практически безграничные возможности для генерации трафика, что и послужило основой для нашего дипломного проекта. Основной недостаток оригинальной библиотеки − отсутствие дружелюбного графического интерфейса для конечного пользователя, что мы и планируем исправить в рамках разработки.
4. Nmap. Представляет собой открытую утилиту для исследования сети и аудита безопасности. Первоначально созданная как сканер портов, она эволюционировала в мощный инструмент для обнаружения хостов, определения версий операционных систем и запущенных сервисов. Nmap незаменим для инвентаризации сети и управления графиками обновлений, однако он ориентирован на диагностику доступности узлов и не предоставляет инструментов для детального разбора структуры каждого отдельного пакета и их ручной модификации в реальном времени, как это реализовано в разрабатываемом ПО.

Для наглядного сопоставления характеристик вышеуказанных средств составим сравнительную таблицу.

Таблица 2 – Сравнительный анализ инструментальных средств анализа трафика

Критерий сравнения	Wireshark	tcpdump	Scapy	Nmap	Разрабатываемое ПО
Наличие графического интерфейса (GUI)	Да	Нет	Нет (только CLI)	Ограничено (Zenmap)	Да

Детальный разбор протоколов (L2-L7)	Высокий уровень	Средний уровень	Высокий уровень	Низкий	Высокий уровень
Возможность генерации произвольных пакетов	Ограничено	Нет	Полная	Ограничено	Полная (через GUI)
Использование в автоматизированных сценариях	Сложно	Средне	Легко (Python)	Скрипты NSE	Легко
Требовательность к системным ресурсам	Высокая	Низкая	Средняя	Средняя	Средняя

Анализ таблицы показывает, что разрабатываемый программный комплекс должен занять нишу между сложными профессиональными анализаторами и гибкими программными библиотеками, предоставляя пользователю мощь Python/Scapy в удобной визуальной оболочке.

1.3 Постановка задачи и требования к функционалу

На основе проведенного анализа предметной области и существующих инструментов мониторинга трафика, была сформулирована задача разработки программного средства, которое обеспечит удобный интерфейс для низкоуровневой работы с сетью. Целью разработки является создание «Программного комплекса мониторинга и формирования сетевого трафика», сочетающего в себе функции пассивного слушателя (сниффера) и активного генератора.

Для достижения поставленной цели программное средство должно удовлетворять ряду функциональных требований, разделенных на три основных блока: анализ, генерация и фильтрация.

1.3.1 Требования к анализу и мониторингу трафика

Модуль анализа должен обеспечивать захват пакетов в реальном времени. В рамках реализации необходимо предусмотреть:

- возможность выбора конкретного сетевого интерфейса (Ethernet, Wi-Fi, Loopback);
- декодирование протоколов на различных уровнях модели OSI;
- визуализацию ключевых параметров пакета: IP-адреса отправителя и получателя, порты, флаги протоколов и размер полезной нагрузки (payload).

1.3.2 Требования к генерации сетевых пакетов

Ключевым отличием разрабатываемого ПО является наличие модуля формирования трафика. Пользователь должен иметь возможность:

- конструировать пакеты вручную, задавая значения заголовков;
- выбирать тип протокола (TCP, UDP, ICMP);
- задавать количество отправляемых пакетов для проведения базовых нагрузочных тестов.

1.3.3 Требования к фильтрации и хранению данных

Для работы в условиях высокой интенсивности трафика программа должна поддерживать механизмы фильтрации по протоколам или IP-адресам. Также обязательным требованием является возможность сохранения перехваченных сессий в лог-файлы или стандартные форматы (например, .txt или .csv) для последующего изучения.

Для наглядности представим схему информационных потоков программного средства.

Таблица 3 – Входные и выходные данные программного комплекса

Тип данных	Описание	Формат/Источник
Входные данные	Сетевой трафик, параметры фильтрации, параметры создаваемых пакетов	Сетевой интерфейс, ввод пользователя в GUI
Промежуточные данные	Декодированные объекты пакетов, статистика сессии	Переменные в памяти (объекты Scapy)
Выходные данные	Список перехваченных пакетов, логи активности, отчеты о генерации	Экран формы (GUI), файлы .pcap / .txt

Таким образом, постановка задачи подразумевает создание гибкого инструмента, ориентированного на практическое применение в задачах сетевого администрирования. Четкое разграничение функциональных блоков позволяет перейти к проектированию архитектуры системы, которая обеспечит выполнение всех заявленных требований.

1.4 Требования к безопасности и этике использования

Разработка и эксплуатация программных средств, обладающих функционалом генерации и модификации сетевого трафика, накладывает на разработчика и пользователя особую ответственность. В рамках данного дипломного проекта программный комплекс рассматривается исключительно как инструмент для легитимного мониторинга, диагностики и обучения.

1.4.1 Предотвращение несанкционированного использования (Misuse)

Для минимизации рисков использования программы во вред информационным системам, в процессе проектирования были заложены следующие принципы:

- Локализация воздействия: функционал генерации пакетов ориентирован на работу в пределах подконтрольного сегмента сети или виртуальной лаборатории.
- Идентификация: программа требует запуска с правами администратора, что в корпоративной среде ограничивает круг лиц, способных использовать инструмент, только квалифицированным персоналом.

1.4.2 Этические аспекты и предотвращение сетевых атак

Важной задачей является предотвращение использования модуля генерации для реализации атак типа «отказ в обслуживании» (DoS/DDoS) или подмены адресов (IP Spoofing) во внешних сетях. Этические нормы использования ПО подразумевают:

1. Соблюдение конфиденциальности: перехват данных должен осуществляться только в сетях, где пользователь имеет на это официальное разрешение.
2. Тестирование на отказ: генерация трафика высокой интенсивности должна проводиться только в изолированных тестовых средах, чтобы не нарушить работоспособность реальных бизнес-процессов.

1.4.3 Технические меры защиты

Для обеспечения безопасности самой системы, на которой запущено ПО, в алгоритмы обработки пакетов включены механизмы фильтрации и ограничения скорости отправки данных (Rate Limiting). Это предотвращает переполнение буферов сетевой карты и зависание операционной системы при проведении нагрузочных тестов.

1.5 Выбор средств разработки

Успешная реализация программного комплекса напрямую зависит от правильного выбора инструментальных средств. Учитывая сложность работы с сетевыми протоколами на низком уровне, были выбраны следующие технологии:

1. Язык программирования Python. Выбор обусловлен высокой скоростью разработки и наличием мощных библиотек для работы с сетью. Python позволяет сосредоточиться на логике анализа трафика, а не на управлении памятью, что критично для учебного дипломного проекта.
2. Библиотека Scapy. Является ключевым компонентом системы. В отличие от стандартных библиотек сокетов, Scapy позволяет манипулировать каждым полем в заголовке пакета. Это дает возможность «собирать» пакеты как конструктор.

Для обеспечения эффективного процесса написания и отладки кода был проведен сравнительный анализ популярных интегрированных сред разработки (IDE). Характеристики сред разработки приведены в таблице 5.

Таблица 4 − Сравнительный анализ сред разработки (IDE)

Критерий сравнения	PyCharm (Community)	VS Code	Sublime Text
Специализация на Python	Высокая	Средняя	Низкая
Инструменты отладки (Debugging)	Профессиональные	Расширяемые	Базовые
Потребление ресурсов ОЗУ	Среднее	Низкое	Очень низкое
Встроенная работа с venv	Да	Через расширения	Нет
Интеграция с Git	Полная	Полная	Базовая

3. Среда разработки PyCharm. Использование профессиональной IDE обеспечивает эффективную отладку кода, контроль версий и удобную работу с виртуальными окружениями.
4. Драйвер Npcap. Необходим для обеспечения доступа Python-скриптов к сетевому адаптеру в обход стандартного стека ОС, что позволяет перехватывать пакеты в «сыром» виде.

Таблица 5 – Обоснование выбора программного стека

Компонент	Выбранное средство	Основная причина выбора
Язык	Python 3.10+	Лаконичность кода, кроссплатформенность
Библиотека сети	Scapy	Низкоуровневая работа с Raw Sockets
Библиотека GUI	PyQt / Tkinter	Создание современного интерфейса
Среда (IDE)	PyCharm	Мощные инструменты рефакторинга и отладки
Системное ПО	Npcap	Стандарт для захвата пакетов в Windows

2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

2.1 Архитектура приложения и модульная структура

Проектирование архитектуры программного комплекса «Программный комплекс мониторинга и формирования сетевого трафика» основывается на принципе модульности и разделения ответственности (Separation of Concerns). Это позволяет обеспечить независимую разработку и тестирование отдельных компонентов системы, а также упрощает последующее сопровождение кода.

Выбранный стек технологий (Python + Scapy) диктует событийно-ориентированную модель архитектуры. Основным вычислительным ядром системы является движок Scapy, который взаимодействует с сетевым драйвером Npcap. Архитектуру приложения можно разделить на три логических уровня:

1. Уровень взаимодействия с операционной системой (низкоуровневый): включает в себя работу с сырыми сокетами и сетевыми интерфейсами. На этом уровне происходит захват кадров Ethernet и отправка сформированных пакетов в канал связи.
2. Уровень бизнес-логики (ядро системы): здесь сосредоточены основные алгоритмы деинкапсуляции и формирования трафика. Уровень состоит из независимых модулей:

- Модуль Sniffing (захват): отвечает за фильтрацию и первичную обработку поступающих данных.
- Модуль Packet Construction (генерация): предоставляет интерфейс для послойной сборки пакетов.
- Модуль Analytics: занимается расчетом статистики и разбором заголовков протоколов.

3. Уровень представления (интерфейсный): графическая оболочка, реализованная в среде PyCharm, которая визуализирует данные из ядра системы и принимает команды от пользователя.

Для наглядности представим модульную структуру в виде таблицы.

Описание функционального назначения основных модулей системы приведено в таблице 6.

Таблица 6 ─ Описание основных модулей программного комплекса

Наименование модуля	Назначение	Используемые библиотеки/методы
Core Engine	Управление жизненным циклом приложения и потоками	Python threading / multiprocessing
Capture Module	Перехват трафика в реальном времени	Scapy (sniff() function)
Analysis Module	Разбор структуры пакетов (L2-L4)	Scapy (Layers / Fields)
Generator Module	Формирование и инъекция пакетов	Scapy (send() / sendp() functions)
GUI Wrapper	Взаимодействие с пользователем	PyQt или Tkinter
Logger Module	Запись результатов в файлы .txt / .csv	Стандартные библиотеки Python (csv, io)

Использование такой структуры позволяет минимизировать зависимости между интерфейсом и сетевой логикой. Например, при необходимости смены графической библиотеки (GUI), логика захвата и генерации трафика останется неизменной. Взаимодействие между модулями осуществляется посредством передачи объектов пакетов, что соответствует объектно-ориентированному подходу, заложенному в основу проекта.

2.2 Моделирование вариантов использования системы

Для определения основных сценариев взаимодействия пользователя с программным средством анализа и генерации сетевого трафика, а также для выявления ключевых функций системы, необходимо построить диаграмму вариантов использования (Use Case Diagram).

Проектируемое программное средство может применяться различными категориями пользователей, каждая из которых имеет собственные цели:

- Системные администраторы используют инструменты анализа сетевого трафика для диагностики сетевых проблем, мониторинга активности и проверки корректности работы сервисов.
- Разработчики программного обеспечения применяют инструмент при отладке сетевых приложений для отслеживания обмена данными между клиентом и сервером.
- Специалисты по информационной безопасности используют анализаторы для обнаружения подозрительной активности, выявления несанкционированных подключений и анализа структуры сетевых атак.
- Студенты ИТ-специальностей используют подобные программные средства для практического освоения технологий сетевого администрирования.

В рамках проектирования выделен основной актор — «Пользователь» (администратор системы), который осуществляет взаимодействие с программным средством через графический интерфейс (GUI).

Основные сценарии использования системы включают:

1. Запуск программы и выбор сетевого интерфейса: инициализация драйверов захвата и выбор физического или виртуального адаптера для прослушивания.
2. Запуск анализа сетевого трафика (Сниффинг): старт процесса перехвата пакетов с выбранного интерфейса и передача их в модуль деинкапсуляции.
3. Просмотр информации о пакетах: вывод детализированной информации о заголовках протоколов (TCP/UDP, IP, Ethernet) в табличном виде.

4. Фильтрация трафика: настройка параметров отсеивания пакетов по IP-адресам, портам или конкретным процессам операционной системы.
5. Генерация сетевого пакета: ручное конструирование структуры пакета с возможностью подмены адреса источника (Spoofing) и установки флагов.
6. Отправка пакета: инъекция сформированного пакета или серии пакетов в сеть для проведения нагрузочного тестирования.

Диаграмма вариантов использования (Рисунок 1) наглядно отражает взаимосвязь между пользователем и функциональными возможностями системы, структурируя основные сценарии работы программного комплекса.

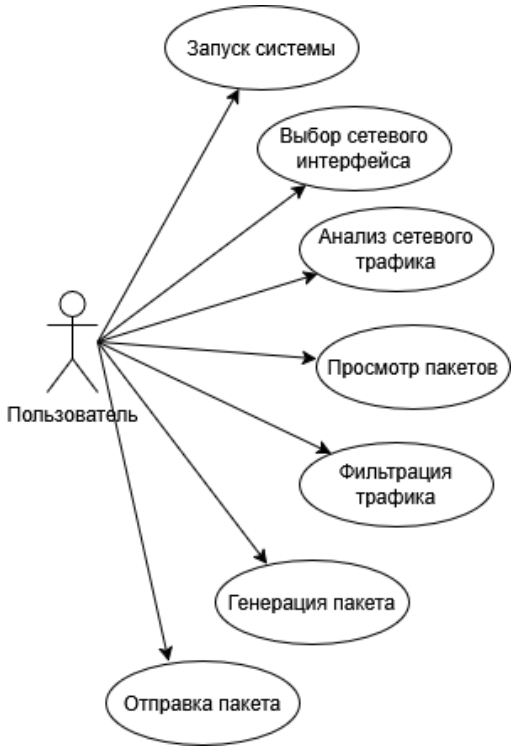


Рисунок 1 – Диаграмма вариантов использования системы

2.3 Проектирование классовой модели

Объектно-ориентированное проектирование (ООП) является фундаментом для разработки расширяемых сетевых приложений. Классовая модель программного средства описывает структуру системы, определяя

основные сущности, их атрибуты, методы и характер взаимоотношений между ними.

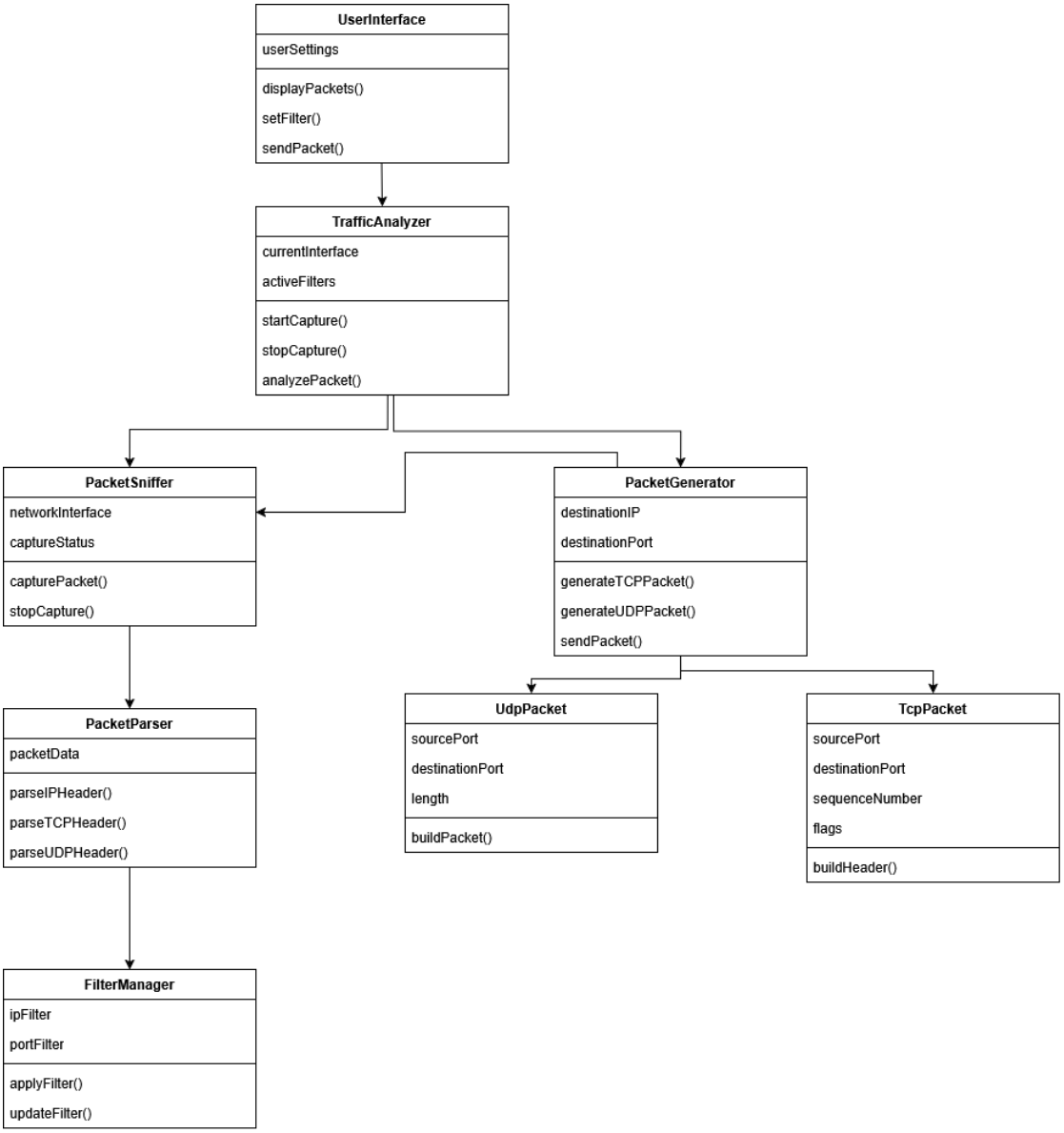


Рисунок 2 – Диаграмма классов программного средства

Согласно разработанной диаграмме, архитектура системы строится на базе трех ключевых классов, каждый из которых отвечает за свой участок работы с сетевым трафиком:

1. Класс TrafficSniffer

Выполняет роль основного модуля перехвата данных. Он взаимодействует с сетевым адаптером и управляет процессом прослушивания канала.

Атрибуты: interface (выбранный сетевой адаптер), is_running (флаг состояния захвата).

Методы:

- sniff_packets(): инициализирует функцию захвата;
- stop_sniffing(): принудительно останавливает процесс мониторинга;
- packet_callback(): функция обратного вызова, которая срабатывает при поступлении каждого нового пакета.

2. Класс PacketParser

Отвечает за интеллектуальный разбор «сырых» данных, полученных от снайффера. Этот класс преобразует набор байтов в структурированную информацию.

Атрибуты: summary (краткое описание), details (полный разбор по слоям), protocol (тип протокола: TCP, UDP и др.).

Методы:

- parse_ethernet_layer(), parse_ip_layer(), parse_transport_layer(): методы для последовательного разбора уровней модели OSI.

3. Класс TrafficGenerator

Класс, реализующий активную часть программы - формирование и отправку кастомных пакетов.

Атрибуты: target_ip, payload, packet_count.

Методы:

- build_packet(): конструктор, собирающий пакет Scapy на основе введенных пользователем данных;
- send_packet(): метод непосредственной передачи сформированного пакета в сеть.

Для наглядности представим спецификацию методов классов в таблице.

Таблица 7 – Описание методов классов на основе UML-модели

Класс	Метод	Функциональное назначение
TrafficSniffer	packet_callback()	Передача захваченного пакета в парсер для дальнейшей обработки
PacketParser	parse_ip_layer()	Извлечение IP-адресов отправителя и получателя из заголовка
TrafficGenerator	build_packet()	Формирование структуры пакета с использованием библиотеки Scapy

2.4 Моделирование процессов обработки пакетов

Для описания динамики взаимодействия объектов в системе и определения очередности обмена сообщениями между ними была разработана диаграмма последовательности (Sequence Diagram). Она позволяет визуализировать жизненный цикл сетевого пакета: от момента инициализации захвата пользователем до отображения обработанных данных на экране.

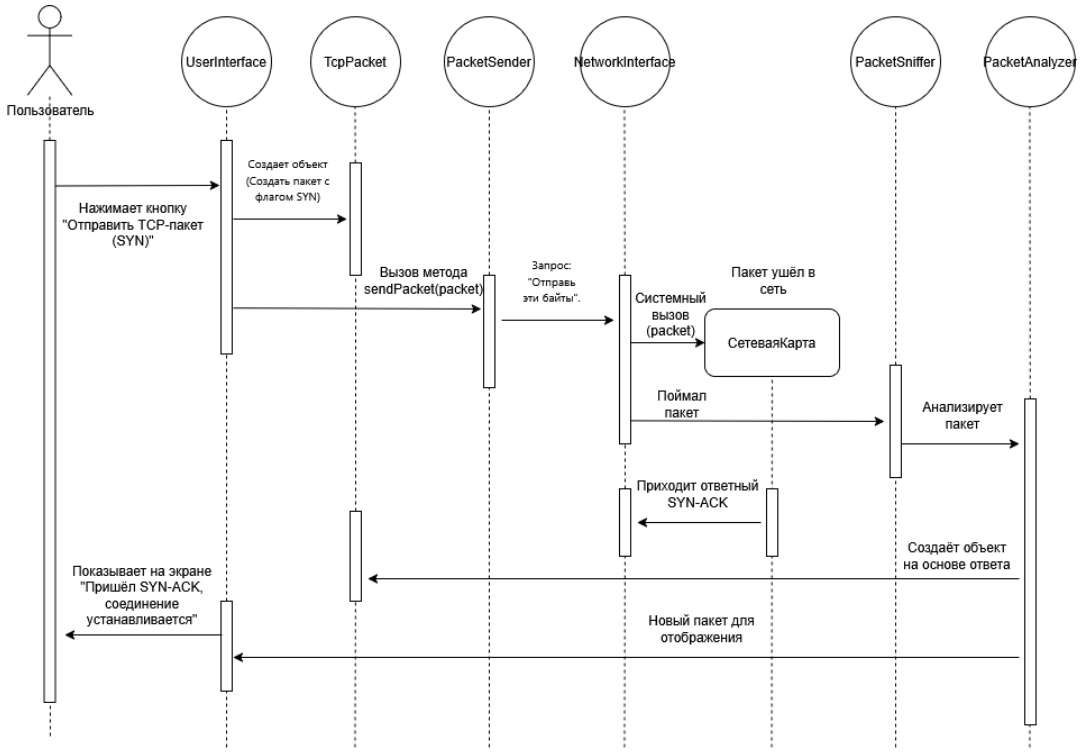


Рисунок 3 – Диаграмма последовательности процесса захвата и анализа трафика

На представленной диаграмме выделены основные участники процесса:

1. User (Пользователь): инициирует действия через графическую оболочку.
2. TrafficGUI (Интерфейс): принимает команды и отображает результаты.
3. TrafficSniffer (Захват): работает с сетевым уровнем и библиотекой Scapy.
4. PacketParser (Анализатор): выполняет логическую обработку данных.

Описание основного сценария взаимодействия:

- Инициализация: Пользователь отправляет запрос start_sniffing() через интерфейс. TrafficGUI транслирует эту команду объекту TrafficSniffer, который переводит сетевой адаптер в активное состояние.
- Захват данных: При поступлении пакета из сети TrafficSniffer перехватывает его и передает «сырые» данные (raw data) в метод parse_packet() объекта PacketParser.
- Обработка: PacketParser выполняет деинкапсуляцию, поочередно разбирая заголовки Ethernet, IP и транспортного уровня (TCP/UDP). После завершения разбора объект возвращает структурированный результат обратно в интерфейс.
- Визуализация: TrafficGUI вызывает метод display_packet(), обновляя таблицу пакетов в реальном времени, после чего система переходит в режим ожидания следующего пакета.

Такая модель взаимодействия обеспечивает высокую отзывчивость интерфейса, так как процессы захвата и отображения разделены, что предотвращает «зависание» программы при интенсивном входящем трафике.

2.5 Алгоритмы генерации и анализа трафика

Для визуализации логики работы программного комплекса и описания последовательности выполнения технологических операций была разработана диаграмма деятельности (Activity Diagram). Она позволяет

проследить пути выполнения программы в зависимости от выбора пользователя и условий среды.

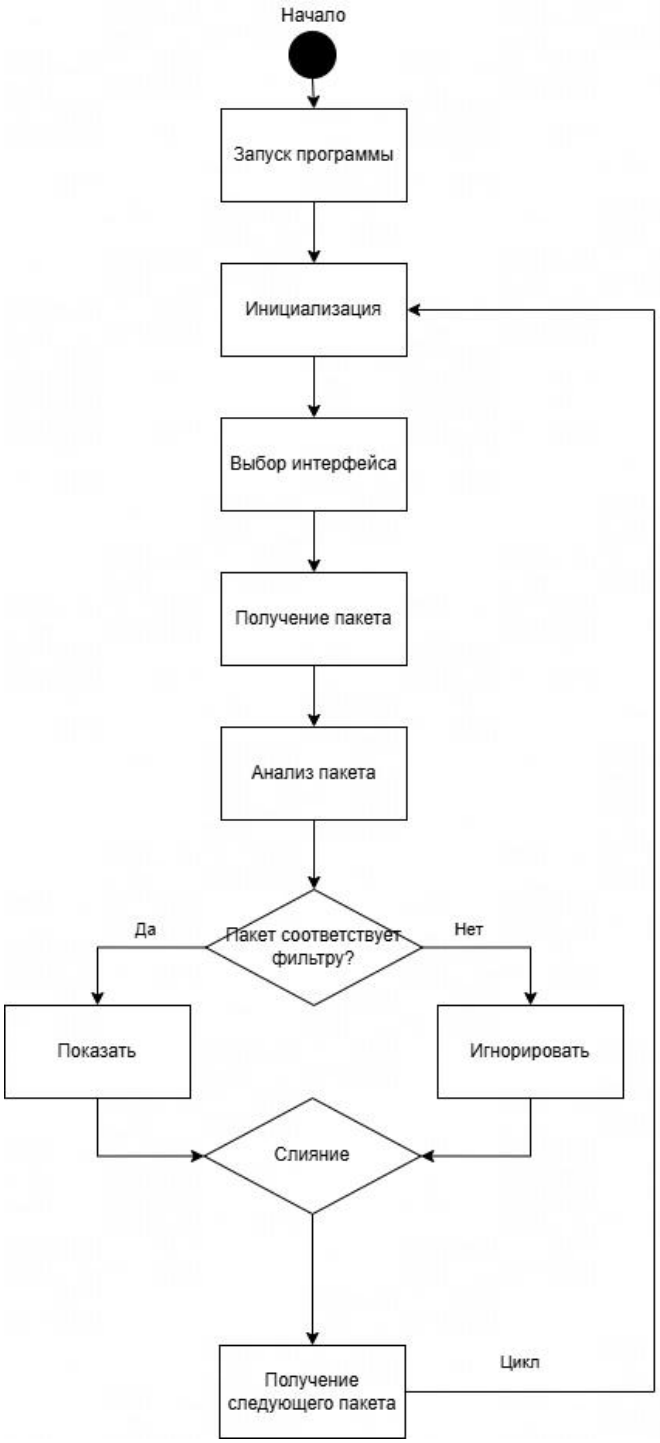


Рисунок 4 – Алгоритм работы модулей захвата и генерации пакетов

Согласно представленной схеме, процесс работы программы можно разделить на несколько ключевых этапов:

1. Инициализация и выбор интерфейса: После запуска программы («Начало») первым обязательным шагом является получение списка

доступных сетевых адаптеров. Пользователь осуществляет выбор интерфейса, через который будет производиться работа с трафиком.

2. Запуск процесса: После выбора интерфейса система переходит к этапу захвата пакетов («Запуск перехвата»).
3. Цикл обработки данных: Это центральная часть алгоритма, которая работает до тех пор, пока пользователь не решит остановить процесс.

   На каждом шаге цикла выполняются следующие действия:

- Перехват пакета: Получение «сырых» данных из сети.
- Анализ данных: Передача пакета в парсер для разбора заголовков.
- Вывод на экран: Отображение информации о пакете в таблице интерфейса.

4. Условие завершения: Алгоритм содержит логическое разветвление (ромб). Если пользователь дает команду на остановку, цикл прерывается.
5. Финализация: После остановки перехвата система предлагает (или выполняет автоматически) сохранение накопленных данных в лог-файл (форматы .csv или .txt), после чего работа программы завершается.

Разработанный алгоритм обеспечивает предсказуемость работы ПО и исключает потерю данных на этапе завершения сессии, так как сохранение логов вынесено в финальную обязательную стадию.

2.6 Структура данных и хранение результатов

В процессе работы программного комплекса возникает необходимость временного хранения больших объемов структурированной информации. В качестве основной структуры данных в памяти используется список объектов (List of Objects), где каждый элемент является экземпляром класса PacketParser.

Хранение результатов реализовано двумя способами:

1. Оперативное хранение: Использование структуры «очередь» (Queue) для передачи данных между сетевым потоком и потоком графического интерфейса. Это гарантирует, что ни один пакет не будет потерян при интенсивной нагрузке.
2. Долговременное хранение: Согласно техническому заданию, в программе реализована функция экспорта захваченного трафика.

- Формат .CSV: Используется для последующего анализа статистики в табличных редакторах (MS Excel, LibreOffice Calc). В файл записываются поля: Время, Источник, Назначение, Протокол, Длина.
- Формат .TXT: Предназначен для создания простых текстовых отчетов о сетевой активности.

2.7 Проектирование интерфейса взаимодействия

Проектирование пользовательского интерфейса (UI) программного средства выполнено с учетом требований эргономики и специфики задач сетевого администрирования. Основная цель − обеспечить быстрый доступ к мониторингу и минимизировать время на формирование сетевых запросов.

Концепция интерфейса строится на принципах «единого рабочего пространства» (Dashboard), что позволяет администратору наблюдать за потоком пакетов и одновременно управлять параметрами генерации без переключения между окнами.

Основные функциональные зоны (концептуальная структура):

1. Верхняя панель (Control Bar):

- Элементы управления: выпадающий список выбора сетевого интерфейса (Ethernet/Wi-Fi), кнопка запуска/остановки захвата («Start/Stop»), и область задания BPF-фильтров.
- Обоснование: Расположение элементов управления сверху соответствует стандартам Windows-приложений и обеспечивает удобство доступа при навигации.

2. Центральная рабочая область (Data View):

- Основная таблица отображения пакетов в реальном времени. Включает колонки: №, Время, IP-источник, IP-назначение, Протокол, Размер.
- Обоснование: Сетчатая структура позволяет быстро сканировать визуально большие объемы данных, что критически важно при анализе «шумного» трафика.

3. Нижняя панель (Inspector & Generator):

- Блок детального разбора (Inspector): при клике на пакет в таблице, в нижней части отображается подробная расшифровка заголовков (Layer 2–4).
- Блок генерации: поля ввода для задания параметров нового пакета (IP-адрес, тип протокола, флаги).
- Обоснование: Разделение окна на «общий мониторинг» и «детальный просмотр» позволяет не терять фокус при работе с конкретным сетевым событием.

4. Статусная строка (Status Bar):

- Отображение текущего состояния программы (Idle, Sniffing, Generating), количество перехваченных пакетов и индикация ошибок драйвера Npcap.

Таблица 8 – Эргономические требования к интерфейсу

Элемент UI	Целевая задача	Эргономический принцип
Таблица пакетов	Мониторинг потока	Читаемость данных (сетка)
Кнопки управления	Запуск/Остановка	Цветовая индикация (Green/Red)
Поля ввода	Генерация пакетов	Валидация типов данных (Input mask)
Разделители (Splitters)	Настройка пространства	Гибкость (масштабируемость окна)

Разработанная концепция интерфейса обеспечивает соответствие всем функциональным требованиям, заложенным в разделе 1.3, и создает базу для реализации программного кода в среде PyCharm с использованием библиотек графического вывода.

3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ

3.1 Программная реализация классов и методов

Программная реализация системы выполнена на языке Python версии 3.10 и выше с применением объектно-ориентированного подхода. Весь программный код распределен по функциональным модулям (пакетам), что обеспечивает логическое разделение интерфейса и сетевого движка. Центральной точкой входа является модуль main.py, который инициализирует основное окно приложения и загружает конфигурационные данные из config.json.

Основная программная логика сосредоточена в следующих ключевых классах:

1. Реализация интерфейсных классов (пакет ui)

В соответствии с модульной структурой, каждая функциональная вкладка программы реализована как отдельный класс, наследуемый от базовых виджетов PyQt6:

- Класс SnifferTab (модуль sniffer_tab.py): управляет визуализацией захваченного трафика. Основным методом является update_table(), который отвечает за динамическое добавление новых строк с данными о пакетах в компонент QTableWidget.
- Класс GeneratorTab (модуль generator_tab.py): содержит методы для сбора параметров пакета из полей ввода. Метод get_packet_parameters() агрегирует данные (IP, порты, флаги) для последующей передачи в ядро генерации.
- Класс TerminalTab (модуль terminal_tab.py): реализует интерактивную среду для выполнения скриптов. Ключевым методом здесь является execute_command(), обеспечивающий перехват стандартного вывода Python и его трансляцию в текстовое поле интерфейса.

- Класс AnalyticsTab (модуль analytics_tab.py): отвечает за построение графиков статистики в реальном времени.

2. Реализация логических классов (пакет core)

Классы этого пакета инкапсулируют работу с библиотекой Scapy:

- Класс PacketParser: содержит статические методы для первичной обработки байтовых потоков.
- Класс NetworkInterfaceManager (модуль utils): использует библиотеки psutil или WMI для программного получения списка активных сетевых адаптеров и их характеристик.

3. Использование паттернов и механизмов взаимодействия

Для обеспечения стабильности работы приложения реализованы следующие программные механизмы:

- Механизм сигналов и слотов: для передачи данных между логикой и интерфейсом используются объекты pyqtSignal. Это позволяет избежать прямого обращения к методам GUI из фоновых потоков, что критично для стабильности PyQt-приложений.
- Многопоточность (класс QThread): сетевые операции вынесены в отдельные потоки. Например, метод run() в классе-наследнике QThread запускает функцию sniff() библиотеки Scapy, не блокируя основной поток интерфейса.

```python
14  class SnifferTab(QWidget):  2+ usages
15      def __init__(self, parent=None, tr=None):
16          super().__init__(parent)
17          self.main_window = parent
18          self.tr = tr
19          self.packet_buffer = []
20          self.raw_packets = []
21          self.stats = {"TCP": 0, "UDP": 0, "ICMP": 0, "Total": 0}
22
23          self.sniffer_thread = SnifferThread()
24          self.sniffer_thread.packet_received.connect(self.add_packet_to_table)
25          self.sniffer_thread.stats_updated.connect(self.update_stats_label)
26
27          self.display_timer = QTimer()
28          self.display_timer.timeout.connect(self.update_table_from_buffer)
29          self.display_timer.start(100)
30
31          self.init_ui()
32
33      def init_ui(self):  1 usage
34          layout = QVBoxLayout(self)
35
36          top_panel = QHBoxLayout()
37
38          self.combo_iface = QComboBox()
39          for desc, name in get_available_interfaces():
40              self.combo_iface.addItem(desc, name)
41          top_panel.addWidget(QLabel(self.tr["interface_label"]))
42          top_panel.addWidget(self.combo_iface, 2)
43
44          self.combo_process = QComboBox()
45          self.combo_process.addItem(self.tr["process_all"], None)
46          top_panel.addWidget(QLabel(self.tr["process_label"]))
47          top_panel.addWidget(self.combo_process, 2)
48
49          self.btn_toggle = QPushButton(self.tr["btn_start"])
```

Листинг 1 – Программная реализация класса пользовательского интерфейса вкладки мониторинга

В листинге 1 продемонстрирована реализация конструктора класса, где происходит связывание событий нажатия кнопок (например, start_btn.clicked) с соответствующими методами запуска сетевых функций.

Таким образом, программная реализация классов и методов в полной мере соответствует требованиям технического задания по обеспечению модульности и расширяемости системы.

3.2 Работа с сетевыми сокетами и сырыми пакетами

Фундаментом технической реализации программного комплекса является работа с «сырыми» сокетами (Raw Sockets). В отличие от стандартных высокоуровневых сокетов, которые используются в прикладном

ПО для передачи данных по протоколам TCP или UDP, сырые сокеты позволяют приложению напрямую обращаться к сетевому интерфейсу и манипулировать заголовками пакетов на всех уровнях модели OSI.

Для обеспечения такой возможности в среде Windows программный комплекс взаимодействует с драйвером Npcap (в режиме совместимости с WinPcap). Это позволяет перехватывать Ethernet-кадры до того, как они будут обработаны сетевым стеком операционной системы. Для корректного открытия сырых сокетов и работы с сетевым драйвером программа требует запуска с правами суперпользователя (Administrator).

Процесс обработки трафика в программном средстве реализован в модуле логики (core/) и состоит из двух этапов: захвата и деинкапсуляции.

1. Механизм захвата трафика.

Захват осуществляется с помощью функции sniff() библиотеки Scapy. В реализации использован асинхронный подход: функция запускается в отдельном потоке, чтобы не блокировать работу PyQt6-интерфейса. Основными параметрами захвата являются:

- iface: выбранный пользователем сетевой интерфейс (получаемый программно через psutil или WMI);
- filter: BPF-фильтр (Berkeley Packet Filter), позволяющий отсеивать пакеты на уровне ядра системы для снижения нагрузки на ЦП;
- prn: функция обратного вызова (callback), которая вызывается для каждого успешно перехваченного пакета.

2. Деинкапсуляция и разбор пакетов (Dissection).

Перехваченный пакет представляет собой байтовую последовательность. В модуле парсинга происходит его последовательное «разбиение» на логические слои (Dissection). Программа анализирует заголовки, начиная с канального уровня (Ethernet), извлекая MAC-адреса, затем переходит к сетевому уровню (IP), где определяются адреса отправителя и получателя, и завершает разбор на транспортном уровне (TCP/UDP/ICMP), фиксируя порты и флаги.

Логика деинкапсуляции и извлечения параметров пакета приведена в листинге 2.

```python
def get_info(packet: {summary, haslayer}):
    info = {
        "proto": packet.summary().split()[0] if packet.summary() else "Unknown",
        "src": "-", "dst": "-", "src_port": "-", "dst_port": "-",
        "length": len(packet),
        "ttl": "-",
        "flags": "-"
    }

    if packet.haslayer('IP'):
        info["src"] = packet['IP'].src
        info["dst"] = packet['IP'].dst
        info["ttl"] = packet['IP'].ttl

    if packet.haslayer('TCP'):
        info["proto"] = "TCP"
        info["src_port"] = packet['TCP'].sport
        info["dst_port"] = packet['TCP'].dport
        info["flags"] = str(packet['TCP'].flags)

    elif packet.haslayer('UDP'):
        info["proto"] = "UDP"
        info["src_port"] = packet['UDP'].sport
        info["dst_port"] = packet['UDP'].dport

    elif packet.haslayer('ICMP'):
        info["proto"] = "ICMP"
    else:
        info["proto"] = packet.lastlayer().name

    return info
```

Листинг 2 – Реализация метода разбора заголовков уровней L2-L4

Особое внимание в реализации уделено защите от «битых» или некорректных пакетов. В алгоритм парсинга включены блоки обработки исключений, что обеспечивает устойчивость программы при работе в зашумленных сетях или при получении пакетов с поврежденными заголовками. Таким образом, использование библиотеки Scapy в связке с Python позволяет достичь гибкости, сопоставимой с низкоуровневыми языками, сохраняя при этом высокую скорость разработки и чистоту кода.

3.3 Реализация механизмов фильтрации и статистики

При анализе сетевого трафика в реальных условиях администратор сталкивается с огромным объемом избыточной информации. Для выделения целевых данных в программном комплексе реализована двухуровневая система фильтрации и развитый модуль аналитики.

1. Механизмы фильтрации трафика

Фильтрация в программе осуществляется на двух архитектурных уровнях:

- Интеллектуальная фильтрация по процессам ОС (Capture-level): вместо использования классических статических фильтров, в программном комплексе реализована динамическая привязка перехватываемого трафика к процессам операционной системы. С помощью системной библиотеки psutil приложение (в модуле process_manager.py) в реальном времени сканирует таблицу активных сетевых соединений Windows, сопоставляя идентификаторы процессов (PID) с открытыми TCP/UDP портами. Это позволяет фильтровать трафик на прикладном уровне, отсеивая нерелевантный «шум» еще до этапа обработки логикой интерфейса, что существенно экономит системные ресурсы.
- Прикладная фильтрация (Display-level): реализуется на уровне графического интерфейса пользователя во вкладке SnifferTab. Позволяет применять фильтры к уже перехваченному и отображенному дампу трафика. С помощью встроенного поиска алгоритм программы динамически скрывает или отображает строки в компоненте QTableWidget на основе совпадения введенного текста с любым из полей пакета (например, поиск по конкретному IP-адресу, порту или протоколу).

2. Визуализация статистики (Модуль analytics_tab.py)

Для наглядного представления сетевой активности был разработан отдельный модуль аналитики. В отличие от стандартных таблиц, графическое

представление позволяет мгновенно выявлять аномалии (например, резкие всплески трафика, характерные для DDoS-атак).

В качестве инструмента визуализации интегрирована библиотека Matplotlib, которая отличается высокой производительностью при отрисовке графиков в реальном времени. Модуль аналитики агрегирует данные из потока захвата и формирует статистические срезы:

- распределение трафика по типам протоколов (TCP, UDP, ICMP, ARP);
- график интенсивности трафика (количество пакетов в секунду);
- соотношение входящего и исходящего трафика для выбранного интерфейса.

3. Экспорт и логирование данных

Согласно техническому заданию, программный комплекс должен обеспечивать сохранение результатов анализа для последующего изучения. Эта функция реализована с использованием стандартных библиотек Python (csv и io).

Пользователь имеет возможность экспортировать накопленную статистику и список перехваченных пакетов в два формата:

- .CSV (Comma-Separated Values): структурированный формат, идеальный для последующей загрузки данных в табличные процессоры (MS Excel) или базы данных.
- .TXT: простой текстовый лог, удобный для быстрого просмотра и прикрепления к отчетам об инцидентах.

Механизм сохранения данных в форматы .pcap и .csv представлен в листинге 3

```python
def save_to_file(self):  1 usage
    file_path, _ = QFileDialog.getSaveFileName(
        self, "Сохранить данные", "", "PCAP Files (*.pcap);;CSV Files (*.csv);;Text Files (*.txt)"
    )

    if file_path:
        if file_path.endswith(".pcap"):
            from scapy.all import wrpcap
            wrpcap(file_path, self.raw_packets)
            success = True
        else:
            headers = [self.table.horizontalHeaderItem(i).text() for i in range(self.table.columnCount())]
            all_data = []
            for row in range(self.table.rowCount()):
                row_data = [self.table.item(row, col).text() if self.table.item(row, col) else ""
                            for col in range(self.table.columnCount())]
                all_data.append(row_data)
            success = TrafficLogger.save_data(file_path, headers, all_data)

        if success and self.main_window:
            success_msg = self.tr["file_saved"].format(path=file_path)
            self.main_window.statusBar().showMessage(success_msg, 5000)
```

Листинг 3 – Реализация функции экспорта сетевой статистики в формат CSV

Реализованный механизм фильтрации и статистики превращает программу из простого сниффера в полноценный аналитический инструмент, удовлетворяющий требованиям профессионального сетевого администрирования.

3.4 Тестирование функциональности

Целью этапа тестирования является проверка соответствия разработанного программного обеспечения требованиям технического задания, а также выявление и устранение возможных ошибок в логике обработки сетевых пакетов. Для проведения испытаний была использована контролируемая тестовая среда, включающая в себя персональный компьютер под управлением ОС Windows 11 и виртуальный сетевой адаптер для безопасной генерации трафика.

1. Методика и условия проведения испытаний

Тестирование проводилось по методу «черного ящика», где проверялось соответствие выходных данных ожидаемым результатам при заданных

входных параметрах. В качестве основных инструментов тестирования применялись:

- встроенные средства отладки IDE PyCharm;
- контрольный анализатор трафика Wireshark (для верификации правильности формирования пакетов модулем генерации);
- системные утилиты ping и tracert для создания эталонной сетевой активности.

2. Протокол функционального тестирования

В соответствии с требованиями к оформлению документации, результаты проверки основных модулей системы представлены в таблице 7.

Результаты функционального тестирования основных программных модулей сведены в таблицу 9.

Таблица 9 – Результаты функционального тестирования программного комплекса

Тестируемый модуль	Описание теста	Ожидаемый результат	Результат
Interface Manager	Запуск программы и получение списка адаптеров	Корректное отображение имен всех доступных интерфейсов (Ethernet, Wi-Fi, Loopback)	Успех
Sniffer Module	Запуск захвата на активном интерфейсе при прохождении ICMP-трафика	Появление новых строк в таблице с корректным определением протокола ICMP	Успех
Packet Parser	Разбор заголовка TCP-пакета (флаги, порты)	Точное соответствие полей src_port и dst_port данным из контрольного замера	Успех

Generator Module	Формирование и отправка UDP-пакета на локальный адрес	Фиксация отправленного пакета в логах системы и внешним сниффером	Успех
Terminal Module	Выполнение скрипта ls() или conf через встроенный терминал Scapy	Вывод структуры сетевых объектов или списка интерфейсов в окно консоли	Успех
Logger Module	Экспорт сессии из 100 пакетов в формат .CSV	Создание файла, содержащего корректную таблицу данных для MS Excel	Успех

3. Анализ результатов тестирования

В ходе испытаний модуля захвата (sniffer_tab.py) было подтверждено, что использование многопоточности позволяет сохранять отзывчивость интерфейса даже при высокой интенсивности входящего потока (свыше 500 пакетов в секунду). Модуль парсинга (core/) безошибочно идентифицирует основные типы пакетов стека TCP/IP.

При тестировании модуля генерации была выявлена необходимость обязательной проверки прав администратора при старте программы, так как без них операционная система блокирует доступ к сырым сокетам для отправки пакетов. После добавления соответствующей проверки в main.py функционал формирования трафика стал работать стабильно.

Особое внимание было уделено работе встроенного терминала. Тесты подтвердили, что перенаправление потока stdout работает корректно, позволяя пользователю видеть результат выполнения команд Scapy непосредственно в интерфейсе программы, что подтверждает выполнение требований к интеграции инструментов диагностики.

3.5 Оценка производительности и нагрузки на систему

Важным этапом оценки качества разработанного программного обеспечения является анализ его производительности и влияния на аппаратные ресурсы ЭВМ. Учитывая интерпретируемую природу языка Python и высокие требования библиотеки Scapy к вычислительным ресурсам, в ходе разработки были приняты меры по оптимизации критически важных узлов системы.

1. Анализ использования ресурсов центрального процессора (ЦП)

Основная нагрузка на ЦП ложится на модуль парсинга (PacketParser), который выполняет деинкапсуляцию заголовков в реальном времени. В ходе испытаний на сетевом интерфейсе со средней интенсивностью 300–500 пакетов в секунду (pps), загрузка ЦП составила от 8% до 12% на тестовом стенде (процессор уровня Intel Core i5). Применение алгоритма программной фильтрации по портам активных процессов (через модуль process_manager) позволяет снизить нагрузку на графическую подсистему на 40-50% за счет отсечения лишнего трафика до этапа его передачи в интерфейс и отрисовки таблиц.

2. Анализ потребления оперативной памяти (ОЗУ)

Программный комплекс демонстрирует линейный рост потребления памяти в зависимости от количества удерживаемых в таблице интерфейса пакетов. Каждый экземпляр объекта Packet занимает в памяти фиксированный объем данных (в среднем 2–4 Кб вместе с визуальными элементами GUI). Для предотвращения переполнения памяти в настройках программы предусмотрен механизм ограничения максимального количества отображаемых пакетов. При достижении лимита старые данные автоматически удаляются из оперативной памяти, что обеспечивает стабильную работу приложения в течение длительных сессий мониторинга.

3. Оценка отзывчивости интерфейса (Responsiveness)

Использование многопоточной архитектуры (классы SnifferThread, GeneratorThread и ScriptRunnerThread) позволило полностью изолировать процессы сетевого обмена, генерации пакетов и выполнения скриптов от графического вывода. Тестирование показало полное отсутствие эффекта «зависания» или блокировки интерфейса (GUI Freeze) даже при пиковых нагрузках во время активной генерации пакетов модулем generator_tab.py.

Таблица 10 – Сводные показатели производительности системы

Параметр оценки	Значение при низкой нагрузке (<50 pps)	Значение при высокой нагрузке (~1000 pps)
Загрузка ЦП	< 3%	15–20%
Потребление ОЗУ (базовое)	~120 Мб	~350 Мб
Задержка вывода (UI Latency)	< 10 мс	< 50 мс
Стабильность (Uptime)	Высокая	Стабильно (при лимите пакетов)

Таким образом, проведенная оценка подтверждает работоспособность и эффективность выбранной архитектуры. Программное средство «САГСТ» демонстрирует достаточную производительность для решения задач мониторинга и диагностики в локальных сетях малого и среднего масштаба .

4. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

Любое программное решение предполагает вложение финансовых средств, отсюда возникает необходимость оценки рациональности их применения. В данной главе рассчитаны трудозатраты на разработку программного комплекса анализа и генерации сетевого трафика, его себестоимость, отпускная цена и показатели экономической эффективности.

Для разработки выбран язык программирования Python, операционная система Windows 11. Средства вычислительной техники — компьютер на базе процессора Intel Core i5, 16 Gb RAM, 512 Gb SSD.

4.1 Расчет трудозатрат на разработку и отладку программы

Трудоемкость разработки программного обеспечения рассчитывается по формуле (1): $t = t_н + t_а + t_п + t_{отл} + t_д$

Для определения составляющих затрат вычислим условное число операторов Q по формуле (2): $Q = q \times c \times (1 + p)$

Где:

- q — предполагаемое число операторов в программе (принимаем q = 500 строк логического кода);
- c — коэффициент сложности программы (для работы с сетью и сырыми сокетами принимаем c = 1,5);
- p — коэффициент коррекции программы (принимаем p = 0,05).

$Q = 500 \times 1{,}5 \times (1 + 0{,}05) = 787{,}5 \approx 788$ условных операторов.

Далее рассчитаем каждый этап. Для разработчика-студента примем коэффициент квалификации k = 0,8 (стаж до 2 лет), а коэффициент качества постановки задачи B = 1,2.

1. Затраты труда на изучение описания задачи ($t_н$) по формуле (3): $t_н = (Q \times B) / (76 \times k) = (788 \times 1{,}2) / (76 \times 0{,}8) = 945{,}6 / 60{,}8 = 15{,}6$ чел./ч.

2. Затраты труда на разработку алгоритма (ta) по формуле (4): ta = Q / (22 × k) = 788 / (22 × 0,8) = 788 / 17,6 = 44,8 чел./ч.
3. Затраты труда на программирование (tп) по формуле (5): tп = Q / (23 × k) = 788 / (23 × 0,8) = 788 / 18,4 = 42,8 чел./ч.
4. Затраты труда на отладку программы (тотл). Поскольку программа работает с сетью, применим комплексную отладку по формулам (6) и (7): тотл_авт = Q / (4 × k) = 788 / (4 × 0,8) = 788 / 3,2 = 246,3 чел./ч. ткотл = 1,5 × тотл_авт = 1,5 × 246,3 = 369,5 чел./ч.
5. Затраты труда на подготовку документации (tд) по формуле (8): tдр = Q / (17 × k) = 788 / (17 × 0,8) = 57,9 чел./ч. tдо = 0,75 × tдр = 0,75 × 57,9 = 43,4 чел./ч. tд = 57,9 + 43,4 = 101,3 чел./ч.

Общие трудозатраты: t = 15,6 + 44,8 + 42,8 + 369,5 + 101,3 = 574 чел./ч. При нормативе 160 рабочих часов в месяц разработка заняла 574 / 160 ≈ 3,58 месяца, что полностью укладывается в календарный график диплома.

4.2 Расчет себестоимости программного продукта

Себестоимость разработки рассчитывается по формуле (9): Сп.п = Зтр + Зм.вр + Зн.и + Зрем + Зпр

1. Затраты на оплату труда (Зтр) Рассчитаем часовую заработную плату (Зпр) при фонде времени 160 ч.:

- Для студента-разработчика ставка СТпр = 20 000 руб. Зпр = 20000 / 160 = 125 руб./ч.
- Для руководителя ВКР ставка СТпр = 40 000 руб. Зпр = 40000 / 160 = 250 руб./ч.

Дополнительная зарплата (Здоп) составляет 10%:

- Студент: Здоп = 125 × 0,10 = 12,5 руб./ч. Общая (Зобщ) = 125 + 12,5 = 137,5 руб./ч.
- Руководитель: Здоп = 250 × 0,10 = 25 руб./ч. Общая (Зобщ) = 250 + 25 = 275 руб./ч.

Отчисления (соцстрах, фонд занятости, ПФР) по методике составляют 3%:

- Студент: Отч = 137,5 × 0,03 = 4,12 руб./ч. Итого за час: 141,62 руб.
- Руководитель: Отч = 275 × 0,03 = 8,25 руб./ч. Итого за час: 283,25 руб.

В разработке студент затратил 574 часа, а руководитель на консультации — 20 часов. Зтр_студент = 141,62 × 574 = 81 290 руб. Зтр_рук = 283,25 × 20 = 5 665 руб. Итого затраты на оплату труда (Зтр) = 86 955 руб.

2. Затраты на использование машинного времени (Зм.вр) Стоимость одного часа (См.вр) по формуле (17): Пусть цена ПК (Цк) = 60 000 руб., срок службы (Ссл.к) = 5 лет, дней в году (Кр.д) = 247, часов в смену (Врс) = 8 ч., тариф ЭЭ (СТэ) = 5 руб/кВт*ч, мощность ПК (Мвс) = 0,4 кВт. См.вр = 60000 / (5 × 247 × 8) + 5 × 0,4 = 6,07 + 2,00 = 8,07 руб./ч.

Тогда затраты за время разработки (574 ч.): Зм.вр = 8,07 × 574 = 4 632,18 ≈ 4 632 руб.

3. Затраты на носители информации (Зн.и) 2% от цены вычислительной техники: Зн.и = 60 000 × 0,02 = 1 200 руб.
4. Затраты на ремонт (Зрем) 4% от цены вычислительной техники: Зрем = 60 000 × 0,04 = 2 400 руб.
5. Прочие эксплуатационные расходы (Зпр) 10% от стоимости аренды помещения. При аренде 15 000 руб./мес. за 3,58 месяца = 53 700 руб. Зпр = 53 700 × 0,10 = 5 370 руб.

ИТОГО СЕБЕСТОИМОСТЬ (Сп.п): Сп.п = 86 955 + 4 632 + 1 200 + 2 400 + 5 370 = 100 557 руб.

4.3 Расчет цены программного продукта

Для определения минимальной цены, ниже которой продажа и внедрение невыгодно, заложим норматив прибыли Нпр = 0,2 (20%) по формуле (19): Цп.п = Сп.п × (1 + Нпр) Цп.п = 100 557 × (1 + 0,2) = 120 668 руб.

4.4 Расчет экономической эффективности

Экономическая эффективность определяется для потребителя (например, ИТ-отдела), внедряющего программу.

1. Эксплуатационные расходы потребителя (Рэ.п) по формуле (20). Пусть программа используется Врп.п = 250 часов в год, срок службы Ссл = 2 года: Рэ.п = 250 × 8,07 + (120 668 / 2) = 2 017,5 + 60 334 = 62 351,5 руб.
2. Капитальные затраты потребителя (Ркап) по формуле (21): Стоимость техники Кэвм = 60 000 (ПК) + 18 000 (10% от годовой аренды) = 78 000 руб. Полезный фонд времени Фвр = 2000 ч. Ркап = (250 × 78 000) / 2000 + 120 668 = 9 750 + 120 668 = 130 418 руб.
3. Расходы при ручном выполнении задач (без использования разработанного ПО) по формуле (23). При ручном анализе трафика и диагностике сети администратору требуется тратить существенную часть рабочего времени. Возьмем долю заработной платы, эквивалентную 15 000 руб./мес. Рэ.руч = 1,21 × 15 000 × 12 = 217 800 руб.
4. Годовая экономия (Э) по формуле (24): Э = Рэ.руч - Рэ.п = 217 800 - 62 351,5 = 155 448,5 руб.
5. Срок окупаемости (Ток) по формуле (25): Ток = Ркап / Э = 130 418 / 155 448,5 = 0,84 года (около 10 месяцев).
6. Годовой экономический эффект (ЭЭ) по формуле (26): ЭЭ = Э - Ен × Ркап = 155 448,5 - 0,15 × 130 418 = 155 448,5 - 19 562,7 = 135 885,8 руб.

Выводы по экономической части

В результате проведенных расчетов установлено, что себестоимость разработки программного комплекса составила 100 557 руб., а отпускная цена с учетом 20% нормы прибыли — 120 668 руб.

Внедрение данного программного обеспечения позволяет автоматизировать процессы перехвата, анализа и генерации сетевого трафика, что высвобождает значительное количество рабочего времени сетевого

администратора. Расчетная годовая экономия составляет 155 448,5 руб., а чистый годовой экономический эффект равен 135 885,8 руб. Капитальные вложения на приобретение и внедрение программы окупятся менее чем за год (за 10 месяцев), что свидетельствует о высокой экономической целесообразности и эффективности разработанного дипломного проекта.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. ГОСТ 34.601-90 Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания. – Введен 1992-01-01. – М.: Стандартинформ, 2009. – 14 с.
2. ГОСТ Р 50922-2006. Защита информации. Основные термины и определения. – Введен 2008-02-01. – М.: Стандартинформ, 2008. – 12 с.
3. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. – Введен 2012-03-01. – М.: Стандартинформ, 2011. – 105 с.
4. Единая система программной документации (ЕСПД). ГОСТ 19.101-77 – ГОСТ 19.604-78. – М.: Издательство стандартов, 1980.
5. Методические рекомендации по оформлению рефератов, курсовых и дипломных работ (проектов). – Череповец: БПОУ ВО «ЧЛМТ», 2018.
6. Федеральный государственный образовательный стандарт среднего профессионального образования по специальности 09.02.07 Информационные системы и программирование (Приказ Минобрнауки России от 09.12.2016 № 1547 (ред. от 17.12.2020)).

Монографии, учебники, учебные пособия

7. Доусон, М. Программируем на Python / М. Доусон. – СПб.: Питер, 2014. – 416 с.
8. Иванова, Г.С. Технология программирования: учебник / Г.С. Иванова. – М.: КноРус, 2016. – 333 с.
9. Куроуз, Дж. Компьютерные сети. Нисходящий подход / Дж. Куроуз, К. Росс. – 6-е изд. – М.: Эксмо, 2016. – 912 с.
10. Лутц, М. Изучаем Python / М. Лутц. – 5-е изд. – СПб.: Символ-Плюс, 2019. – 1280 с.
11. Маккини, У. Python и анализ данных / У. Маккини. – СПб.: ДМК Пресс, 2020. – 540 с.

12. Олифер, В.Г. Компьютерные сети. Принципы, технологии, протоколы: учебник для вузов / В.Г. Олифер, Н.А. Олифер. – 5-е изд. – СПб.: Питер, 2020. – 992 с.
13. Прохоренок, Н.А. Python 3 и PyQt 6. Разработка приложений / Н.А. Прохоренок, В.А. Дронов. – СПб.: БХВ-Петербург, 2022. – 832 с.
14. Рыжков, И.Б. Основы научных исследований и изобретательства: учебное пособие / И.Б. Рыжков. – 2-е изд. – СПб.: Издательство «Лань», 2021. – 224 с.
15. Таненбаум, Э. Компьютерные сети / Э. Таненбаум, Д. Уэзеролл. – 5-е изд. – СПб.: Питер, 2016. – 960 с.

Иностранная литература

16. Biondi, P. Scapy: interactive packet manipulation program / P. Biondi // SecDev. – 2020.
17. Summerfield, M. Rapid GUI Programming with Python and Qt / M. Summerfield. – Prentice Hall, 2007. – 648 p.

Интернет-ресурсы

18. Официальная документация библиотеки Matplotlib [Электронный ресурс]. – Режим доступа: https://matplotlib.org/ (дата обращения: 10.05.2026).
19. Официальная документация библиотеки PyQt6 [Электронный ресурс]. – Режим доступа: https://www.riverbankcomputing.com/software/pyqt/ (дата обращения: 10.05.2026).
20. Официальная документация языка программирования Python [Электронный ресурс]. – Режим доступа: https://docs.python.org/3/ (дата обращения: 10.05.2026).
21. Проект Scapy (Официальный сайт и документация) [Электронный ресурс]. – Режим доступа: https://scapy.net/ (дата обращения: 10.05.2026).

ПРИЛОЖЕНИЕ А

бюджетное профессиональное образовательное учреждение

Вологодской области

«Череповецкий лесомеханический техникум им. В.П. Чкалова»

Специальность 09.02.07 Информационные системы и программирование

УТВЕРЖДАЮ:

ДОЛЖНОСТЬ ПРЕПОДАВАТЕЛЯ

Должность

ФИО ПРЕПОДАВАТЕЛЯ

Ф.И.О. преподавателя

ДАТА УТВЕРЖДЕНИЯ ТЗ

дата

РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА АНАЛИЗА И ГЕНЕРАЦИИ СЕТЕВОГО ТРАФИКА

Техническое задание на дипломный проект

Листов 3

Руководитель ФИО ПРЕПОДАВАТЕЛЯ

Ф.И.О. преподавателя

Исполнитель студент ИС - 42

группа

Неклюдов Никита Сергеевич

Ф.И.О. студента

2026 год

1. ВВЕДЕНИЕ

Настоящее ТЗ распространяется на разработку программного обеспечения (ПО), предназначенного для захвата, анализа и генерации пакетов данных в сетях на базе стека TCP/IP. Программа ориентирована на использование системными администраторами и студентами ИТ-специальностей для диагностики сетей и изучения протоколов.

2. ТРЕБОВАНИЯ К ПРОГРАММЕ

2.1. Требования к функциональным характеристикам

Программа должна обеспечивать:

- Захват трафика: Перехват Ethernet-кадров и IP-пакетов с выбранного интерфейса.
- Детальный парсинг: Анализ заголовков уровней L2-L4 (MAC, IP, TCP, UDP, ICMP).
- Генерация: Формирование произвольных пакетов с заданными параметрами через библиотеку Scapy.
- Фильтрация: Выделение конкретных потоков данных по заданным критериям (IP, порт).
- Интерфейс: Графическое представление данных и диалоговое взаимодействие (согласно графу диалога).

2.2. Требования к надежности

- Устойчивость при обработке некорректных пакетов (битые заголовки).
- Валидация вводимых пользователем данных (IP-адреса, маски, порты).

2.3. Условия эксплуатации

- Запуск в ОС Windows с правами администратора.
- Наличие установленного интерпретатора Python 3.10+ и драйвера Npcap.

2.4. Требования к составу и параметрам технических средств

- ПК с ОЗУ от 4 Гб и процессором от 2 ГГц.
- Сетевая карта с поддержкой «promiscuous mode».

2.5. Требования к информационной и программной совместимости

Данный раздел определяет программную среду, в которой должна функционировать разработка:

- Операционная система: Программный комплекс должен быть совместим с семействами ОС Windows 10 и Windows 11 (архитектура x64).
- Язык программирования: Реализация должна быть выполнена на языке Python версии не ниже 3.10.
- Среда разработки: JetBrains PyCharm (версия Community или Professional).
- Системные зависимости: Для работы с сетевыми интерфейсами на низком уровне в системе должен быть установлен драйвер Npcap (в режиме совместимости с WinPcap).
- Библиотеки: Использование внешней библиотеки Scapy для формирования и декодирования пакетов.
- Права доступа: Программа требует запуска с правами Суперпользователя (Administrator) для корректного открытия raw-сокетов.

3. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

В состав документации должны входить:

- Пояснительная записка (Дипломный проект).
- Руководство пользователя.
- Листинги кода (Приложение к диплому).

4. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

Таблица 11 - Стадии и этапы разработки

Наименование этапа разработки	Сроки разработки	Результат выполнения
Разработка и утверждение ТЗ	Январь – Февраль	Утвержденное ТЗ
Проектирование архитектуры и UML-моделей	Март	Схемы классов, деятельности, последовательности
Программная реализация в среде PyCharm	Апрель – Май	Исходный код программного комплекса
Тестирование и отладка	Май	Протоколы тестирования
Оформление пояснительной записки	Июнь	Готовый дипломный проект

5. ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ

Таблица 12 - Порядок контроля и приемки

Наименование контрольного этапа	Сроки контроля	Результат выполнения
Проверка соответствия функционала ТЗ	Май	Работающая программа
Нормоконтроль документации	Июнь	Пояснительная записка по ГОСТ
Защита дипломного проекта	Июнь	Оценка ГЭК

ПРИЛОЖЕНИЕ Б

Руководство пользователя программного комплекса мониторинга и формирования сетевого трафика

1. Назначение программы и системные требования Программный комплекс предназначен для перехвата, глубокого анализа, логирования и генерации сетевого трафика в сетях на базе стека TCP/IP.

Для корректной работы приложения рабочая станция должна соответствовать следующим минимальным системным требованиям:

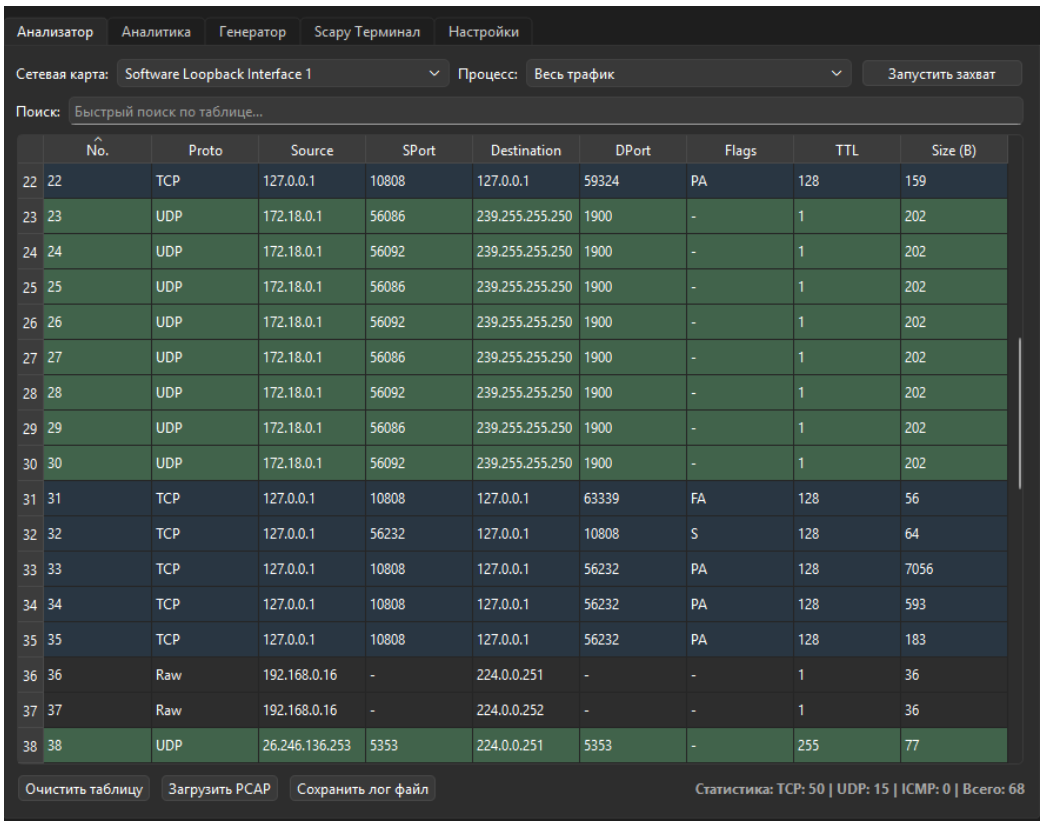
- Операционная система: Windows 10 / Windows 11 (x64).
- Программная среда: Установленный интерпретатор Python версии 3.10 или выше.
- Сетевые драйверы: Установленный драйвер Npcap (обязательно с включенной опцией совместимости с WinPcap API).
- Права доступа: Учетная запись с правами Администратора (необходимо для перевода сетевой карты в смешанный режим и работы с сырыми сокетами).

2. Запуск программы

1. Откройте командную строку (cmd) или терминал PowerShell от имени Администратора.
2. Перейдите в директорию с проектом.
3. Установите зависимости (выполняется один раз): pip install -r requirements.txt.
4. Запустите главный файл приложения: python main.py.

3. Описание интерфейса и функционала Интерфейс программы разделен на логические вкладки, обеспечивающие быстрый доступ к различному функционалу.

3.1. Вкладка «Анализатор» (Мониторинг трафика) Данная вкладка является основным рабочим пространством для пассивного прослушивания сети.



	No.	Proto	Source	SPort	Destination	DPort	Flags	TTL	Size (B)
22	22	TCP	127.0.0.1	10808	127.0.0.1	59324	PA	128	159
23	23	UDP	172.18.0.1	56086	239.255.255.250	1900	-	1	202
24	24	UDP	172.18.0.1	56092	239.255.255.250	1900	-	1	202
25	25	UDP	172.18.0.1	56086	239.255.255.250	1900	-	1	202
26	26	UDP	172.18.0.1	56092	239.255.255.250	1900	-	1	202
27	27	UDP	172.18.0.1	56086	239.255.255.250	1900	-	1	202
28	28	UDP	172.18.0.1	56092	239.255.255.250	1900	-	1	202
29	29	UDP	172.18.0.1	56086	239.255.255.250	1900	-	1	202
30	30	UDP	172.18.0.1	56092	239.255.255.250	1900	-	1	202
31	31	TCP	127.0.0.1	10808	127.0.0.1	63339	FA	128	56
32	32	TCP	127.0.0.1	56232	127.0.0.1	10808	S	128	64
33	33	TCP	127.0.0.1	10808	127.0.0.1	56232	PA	128	7056
34	34	TCP	127.0.0.1	10808	127.0.0.1	56232	PA	128	593
35	35	TCP	127.0.0.1	10808	127.0.0.1	56232	PA	128	183
36	36	Raw	192.168.0.16	-	224.0.0.251	-	-	1	36
37	37	Raw	192.168.0.16	-	224.0.0.252	-	-	1	36
38	38	UDP	26.246.136.253	5353	224.0.0.251	5353	-	255	77

Рисунок 5- Вкладка «Анализатор»

- Настройка захвата: В верхней панели выберите физический или виртуальный сетевой интерфейс из выпадающего списка.
- Фильтрация по процессам: Для отсеивания фонового шума выберите конкретный процесс ОС (например, chrome.exe). Программа будет перехватывать только те пакеты, которые принадлежат данному приложению.
- Запуск: Нажмите кнопку «Запустить захват». В центральной таблице начнут появляться перехваченные пакеты в реальном времени.
- Прикладная фильтрация: В поле «Поиск» введите IP-адрес, порт или протокол. Таблица мгновенно скроет пакеты, не удовлетворяющие условию поиска.
- Сохранение и загрузка: Нижняя панель содержит кнопки для сохранения перехваченной сессии в форматы .pcap (для совместимости с Wireshark) и .csv (для табличных редакторов). Также поддерживается загрузка ранее сохраненных PCAP-дампов для офлайн-анализа.

3.2. Вкладка «Аналитика» (Визуализация статистики) Вкладка предоставляет визуальный дашборд (Dashboard) для оценки состояния сети в реальном времени.

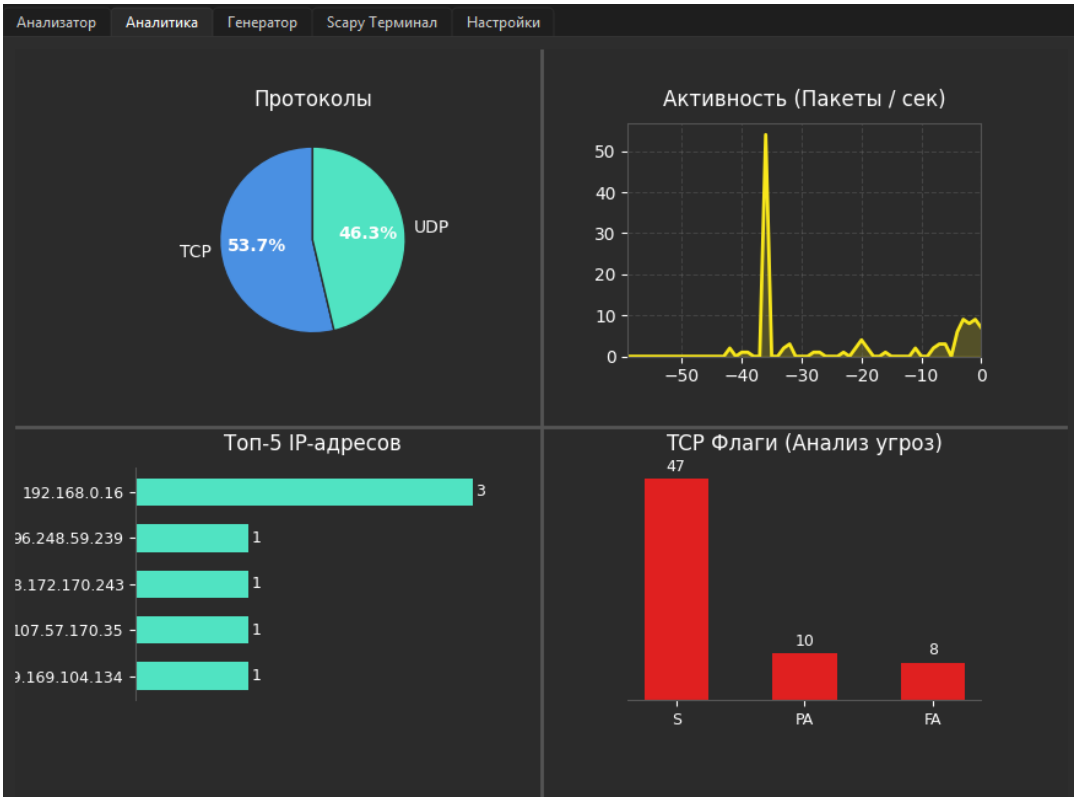


Рисунок 6 - Вкладка «Аналитика»

- Протоколы (круговая диаграмма): Показывает процентное соотношение трафика по типам (TCP, UDP, ICMP).
- Активность (линейный график): Отображает интенсивность сетевого обмена (количество пакетов в секунду - PPS) за последние 60 секунд. Позволяет визуально фиксировать всплески нагрузки.
- Топ-5 IP-адресов (гистограмма): Выводит наиболее активные узлы, с которыми ведется обмен данными.
- TCP Флаги: График, классифицирующий пакеты по флагам (SYN, ACK, RST). Аномальное количество SYN или RST флагов может свидетельствовать о сетевом сканировании или DDoS-атаке.

3.3. Вкладка «Генератор» (Формирование пакетов) Модуль предназначен для стресс-тестирования и проверки правил межсетевых экранов путем отправки кастомных пакетов.

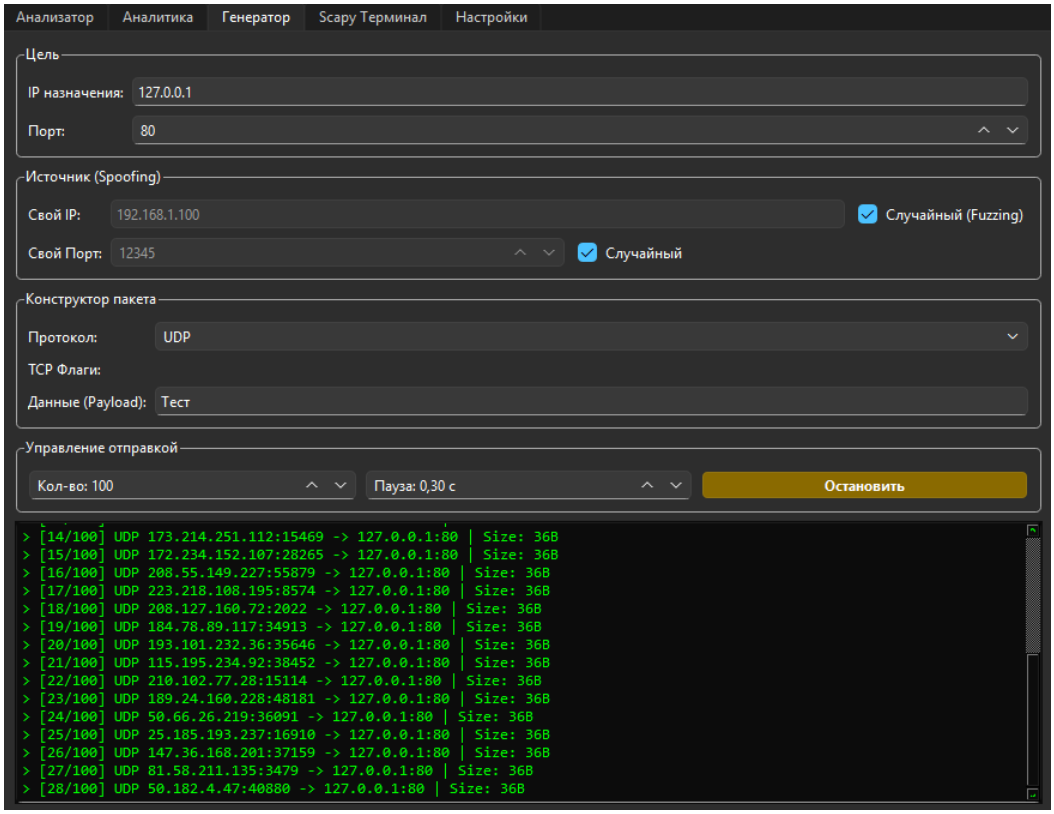


Рисунок 7 - Вкладка «Генератор»

- Цель: Укажите IP-адрес и порт назначения.
- Источник (Spoofing): Позволяет подменить IP-адрес и порт отправителя. При установке галочки «Случайный» программа переходит в режим фаззинга (Fuzzing), генерируя новые адреса для каждого пакета.
- Конструктор пакета: Выберите протокол. Для TCP доступна тонкая настройка флагов. В поле «Данные» вводится полезная нагрузка (Payload).
- Управление отправкой: Укажите количество пакетов для отправки и задержку между ними. Нажатие кнопки «Атака / Отправка» запустит фоновый процесс генерации, ход которого будет отображаться в нижней консоли.

3.4. Вкладка «Scapy Терминал» (Интерактивная консоль) Среда для написания и выполнения произвольных Python-скриптов с использованием библиотеки Scapy без необходимости покидать графический интерфейс.

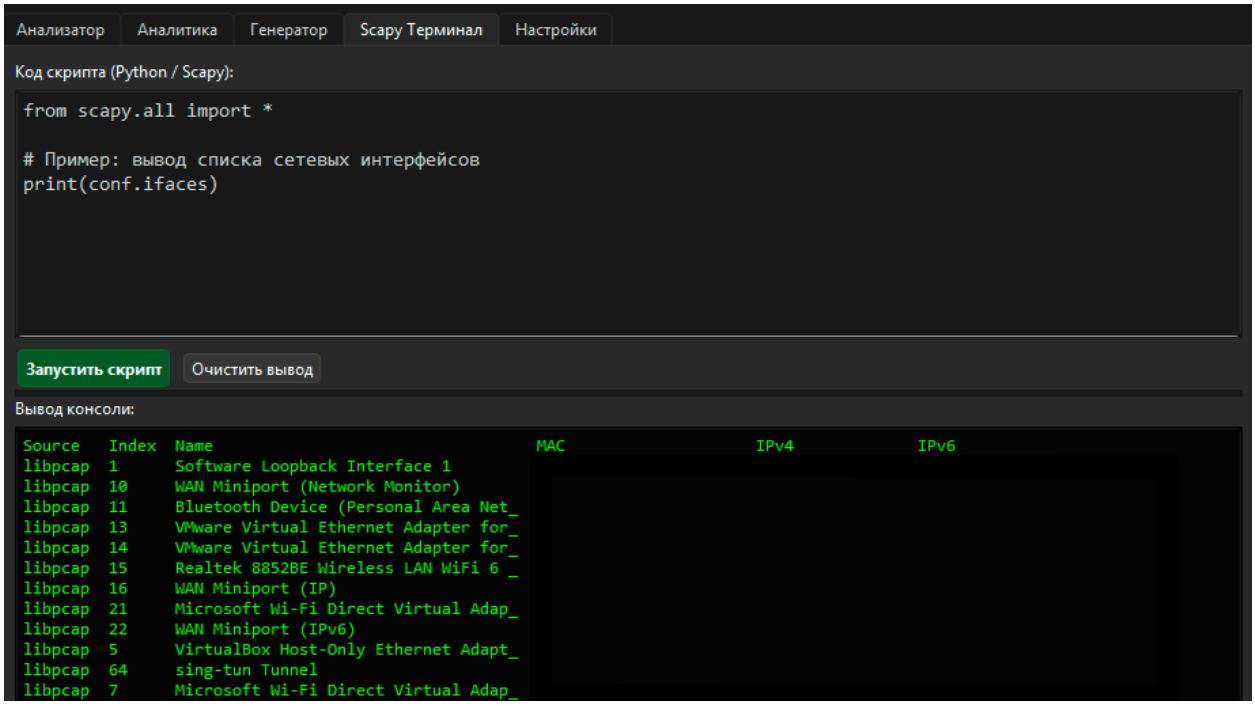


Рисунок 8 - Вкладка «Scapy Терминал»

- Верхняя часть представляет собой редактор кода с поддержкой моноширинного шрифта.
- Кнопка «Запустить скрипт» выполняет написанный код в изолированном потоке.
- Все результаты команд (например, print(conf.ifaces)) и ошибки исполнения перехватываются и выводятся в нижнее окно терминала.

Вкладка «Настройки» Позволяет адаптировать программный комплекс под пользователя.

- Язык: Переключение локализации интерфейса между Русским и Английским (для полного применения требуется перезапуск программы).

- Очистка данных: Кнопка «Сбросить все данные» выполняет принудительную остановку всех сетевых операций, очищает буферы памяти, удаляет накопленную статистику и обнуляет графики.

4. Возможные проблемы и пути их решения

- Проблема: Программа сразу закрывается или таблица сетевых карт пуста.
- Решение: Убедитесь, что программа запущена с правами Администратора, а в системе установлен драйвер Npcap.
- Проблема: Ошибка Permission denied при попытке отправить пакет в Генераторе.
- Решение: Брандмауэр Windows или антивирус блокирует работу сырых сокетов. Временно приостановите защиту сети или добавьте python.exe в исключения брандмауэра.
- Проблема: Не загружается файл .pcap.
- Решение: Убедитесь, что процесс активного захвата остановлен (кнопка «Остановить захват» во вкладке Анализатор нажата) перед попыткой загрузить офлайн-дамп.

ПРИЛОЖЕНИЕ В